



CS61C

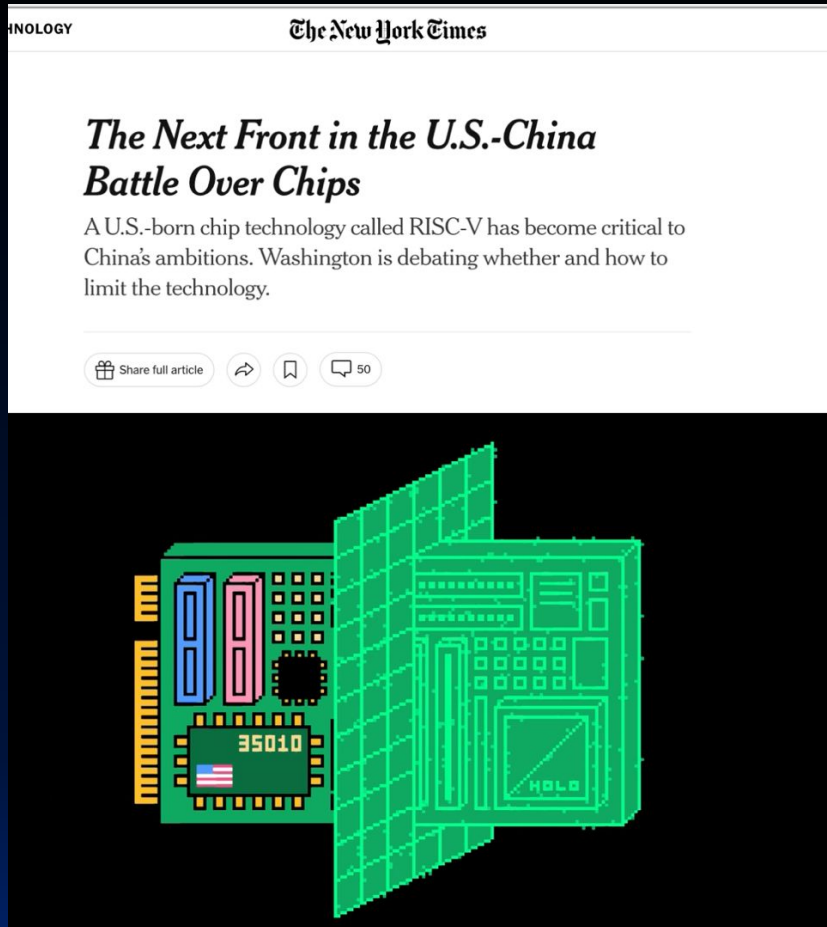
Great Ideas
in
Computer Architecture
(a.k.a. Machine Structures)



Assistant
Teaching Professor
Lisa Yan

Running a Program – CALL (Compiling, Assembling, Linking, and Loading)





The debate over RISC-V is complicated because the technology was patterned after [open-source software](#), the free programs like Linux that allow any developer to view and modify the original code used to make them. Such programs have prompted multiple competitors to innovate and reduce the market power of any single vendor.

But RISC-V is not code that can directly be used to make anything. It is a set of basic computing instructions that determine the calculations a chip can perform. Engineers can download these instructions and incorporate them in the much more complex task of creating design blueprints for parts of a semiconductor. Many companies sell RISC-V chip designs, and some universities and other institutions distribute them free.

U.S. regulations limit Arm and RISC-V companies from exporting chip designs to China based on certain performance limits. But trying to restrict the underlying instructions is like trying to control words or letters, Silicon Valley executives said.

Great Idea #1: Abstraction (Levels of Representation/Interpretation)

How do we translate programs from a high-level language into machine code?

High Level Language
Program (e.g., C)

```
temp = v[k];
v[k] = v[k+1];
v[k+1] = temp;
```

Compiler

Assembly Language
Program (e.g., RISC-V)

```
lw    x3, 0(x10)
lw    x4, 4(x10)
sw    x4, 0(x10)
sw    x3, 4(x10)
```

Assembler

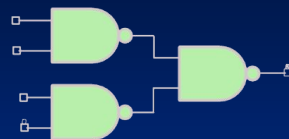
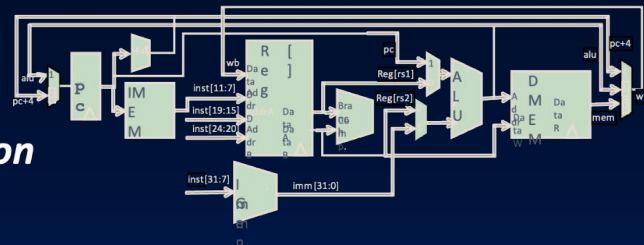
Machine Language
Program (RISC-V)

```
1000 1101 1110 0010 0000 0000 0000 0000
1000 1110 0001 0000 0000 0000 0000 0100
1010 1110 0001 0010 0000 0000 0000 0000
1010 1101 1110 0010 0000 0000 0000 0100
```

Hardware Architecture Description
(e.g., block diagrams)

Architecture Implementation

Logic Circuit Description
(Circuit Schematic Diagrams)



"Compilation" vs. Translation

[Review]

C program file: foo.c

"Compile"
(colloquially)

Executable (mach. lang. program): a.out

Loader

Memory

- "Compiling C code to binary" generally/**colloquially** refers to translating a C program to an executable.
- So far: `gcc foo.c`
...produces `a.out`.

CALL: Compile, Assemble, Link, Load

Lots of details to **recall!**
Today's lecture builds **intuition** and avoids details. Read the notes!

- CALL: Compile, Assemble, Link, Load
- [Example] Hello, World!
- [Aside] Static vs. Dynamically Linked Libraries

Steps in Compiling and Running a C Program

C program file: foo.c

Compiler

Assembly program file: foo.s

Assembler

Object file (mach. lang. module): foo.o

Linker

Executable (mach. lang. program): a.out

Loader

Memory

- “**Compiling C code to binary**” generally/**colloquially** refers to translating a C program to an executable.
- So far: `gcc foo.c`
...produces `a.out`.
- This translation process is actually **3 steps**:
 - Compile, Assemble, Link
 - ...followed by Load to execute

Step 1: Compiler

C program file: **foo.c**

Compiler

Assembly program file: **foo.s**

Assembler

Object file (mach. lang. module): **foo.o**

Linker

Executable (mach. lang. program) **a.out**

Loader

Memory

- Input: High-level Language Code (e.g., **foo.c**)
- Output: Assembly Language Code (e.g., **foo.s** for RISC-V)
- Note: Output may contain **pseudoinstructions!**
 - e.g., **mv, li, call, j, etc.**



Step 2: Assembler

gcc -c foo.c

C program file: foo.c

Compiler

Assembly program file: foo.s

Assembler

Object file (mach. lang. module): foo.o

lib.o ➔ Linker

Executable (mach. lang. program) a.out

Loader

Memory

- Input: Assembly Language Code (e.g., **foo.s** for RISC-V)
- Output: Machine Language Module, object file (e.g., **foo.o** for RISC-V)

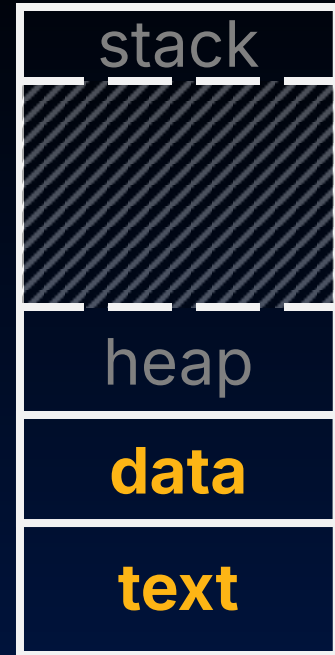
The assembler doesn't know where headers are located; it delegates this resolution to the Linker, downstream.

```
#include <stdio.h>
int main() {
    printf("Hello, %s\n",
           "world");
    return 0;
}
```




Assembler builds Text and Data Segments

- **Replace pseudoinstructions** with true assembly instructions.
- **Produce an object file** (a machine language **module**) that contains both this module's instructions and information for downstream steps (e.g., linking) and debugging.
 - Text segment
 - Data segment (e.g., static variables)
- Resolves **position-independent code**
 - PC-Relative Branches and Jumps, e.g., `beq/bne/etc.` and `jal`



Assembler defers to downstream (Linker) any references that depend on knowing **other** object modules.



Step 3: Linker

gcc foo.o

C program file: foo.c

Compiler

Assembly program file: foo.s

Assembler

Object file (mach. lang. module): foo.o

lib.o ➡ Linker

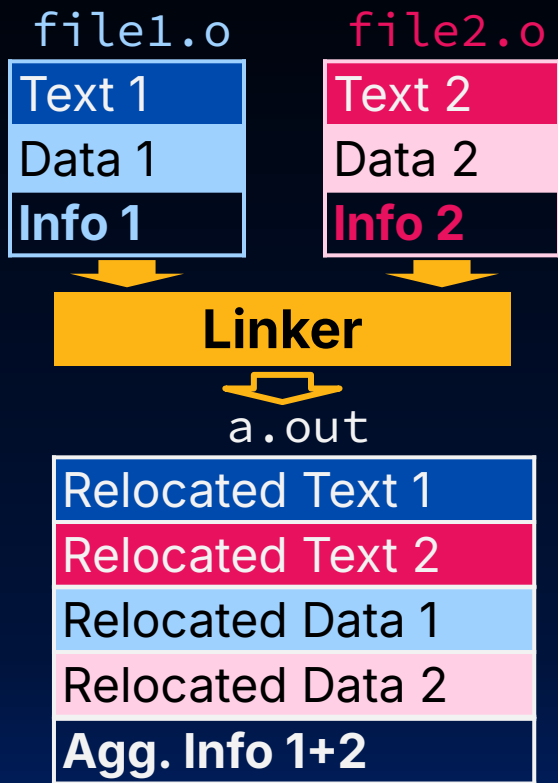
Executable (mach. lang. program): a.out

Loader

Memory

- Input: Object files
(e.g., **foo.o**, **lib.o** for RISC-V)
- Output: Executable machine code
(e.g., **a.out** for RISC-V)
- The Linker enables **separate** compilation of files.
 - Changes to one file **does not require recompilation** of the entire program.
 - C standard libraries, e.g., stdio (e.g., Linux source >20M lines of code)

Linker Patches Together Multiple Object Modules



1. Put together **text segments** from each .o file.
2. Put together **data segments** from each .o file, then concatenate this onto the end of Step 1's segment.
3. Resolve references.
 - Go update all **absolute addresses** to external labels and static data.

The linker knows all input object file modules, so it can resolve all the assembler's "TODO items" that require relocating for the final executable.

Info: symbol table, relocation table, etc.

Step 4: Loader

C program file: foo.c

Compiler

Assembly program file: foo.s

Assembler

Object file (mach. lang. module): foo.o

Linker

lib.o

Executable (mach. lang. program): a.out

Loader

Memory

- Input: Executable Code
(e.g., **a.out** for RISC-V)
 - Executable files are stored on disk.
- Output: (program is run)
 - When an executable is run, the loader first loads it into memory and then starts it running.

In reality: Loading is **an Operating System (OS) task**. Take CS162 for more!



Peer Instruction

At what point in the process are all the **machine code bits** determined for the following assembly instructions?

1. `add x6 x7 x8`

A. After compilation

B. After assembly

C. After linking

D. After loading

2. `jal x1 fprintf`

A. After compilation

B. After assembly

C. After linking

D. After loading

L18: At what point in the process are all the machine code bits determined?

At what point in the process are all the **machine code bits** determined for the following assembly instructions?

1. `add x6 x7 x8`

A. After compilation

B. After assembly

C. After linking

D. After loading

2. `jal x1 fprintf`

A. After compilation

B. After assembly

C. After linking

D. After loading





Peer Instruction: Answer

At what point in the process are all the **machine code bits** determined for the following assembly instructions?

1. `add x6 x7 x8`

A. After compilation

B. After assembly

C. After linking

D. After loading

2. `jal x1 fprintf`

A. After compilation

B. After assembly

C. After linking

D. After loading

`fprintf` is
an external
function (C
`stdio`)

Yan, SP26



Agenda

[Example] Hello, World!

- CALL: Compile, Assemble, Link, Load
- [Example] Hello, World!
- [Aside] Static vs. Dynamically Linked Libraries



Example C Program: hello.c

```
#include <stdio.h>
int main() {
    printf("Hello, %s\n",
           "world");
    return 0;
}
```



Step 1. Compile: `hello.c` → `hello.s`

```
.text
.align 2
.globl main
main:
    addi sp, sp, -4
    sw    ra, 0(sp)
    la    a0, str1
    la    a1, str2
    call  printf
    lw    ra, 0(sp)
    addi  sp, sp, 4
    li    a0, 0
    ret
.section .rodata
.balign 4
str1:
.string "Hello, %s!\n"
str2:
.string "world"
```

```
#include <stdio.h>
int main() {
    printf("Hello, %s\n",
           "world");
    return 0;
}
```

Step 1. Compile: Pseudoinstructions

```
.text
.align 2
.globl main
main:
    addi sp,sp,-4
    sw   ra,0(sp)
    la   a0, str1
    la   a1, str2
    call printf
    lw   ra,0(sp)
    addi sp,sp,4
    li   a0,0
    ret
.section .rodata
.balign 4
str1:
.string "Hello, %s!\n"
str2:
.string "world"
```

} **pseudoinstructions**

```
#include <stdio.h>
int main() {
    printf("Hello, %s\n",
           "world");
    return 0;
}
```



Step 1. Compile: Specify Directives

```
.text
.align 2
.globl main
main:
    addi sp sp -4
    sw   ra 0(sp)
    la   a0 str1
    la   a1 str2
    call printf
    lw   ra 0(sp)
    addi sp sp 4
    li   a0 0
    ret
```

Enter **text** section
Align code to **2*2** bytes
Declare global symbol **main**
(all three are directives)

```
#include <stdio.h>
int main() {
    printf("Hello, %s\n",
           "world");
    return 0;
}
```

```
.section .rodata
.balign 4
```

Directive: Enter **read-only data** section
Directive: Align data section to **4** bytes

```
str1:
.string "Hello, %s!\n"
str2:
.string "world"
```

Label
Directive: null-terminated string
Label
Directive: null-terminated string



Step 2. Assemble: `hello.s` → `hello.o`

Text segment

```
00000000 <main>:
0:  ff010113 addi  sp  sp  -16
4:  00112623 sw    ra  12(sp)
8:  00000537  lui    a0  0x0
c:  00050513  addi   a0  a0  0
10: 000005b7  lui    a1  0x0
14: 00058593  addi   a1  a1  0
18: 000080e7  jalr   ra  0
1c: 00c12083 lw     ra  12(sp)
20: 01010113 addi   sp  sp  16
24: 00000513 addi   a0  a0  0
28: 00008067 jalr   ra
```

} la x2, call
pseudo-
instructions
replaced

Address
placeholders
underlined

Object file (Assembler output)

```
... ff010113 00112623 00000537
00050513 000005b7 00058593 000080e7
00c12083 01010113 00000513 00008067 ...
```

Step 2. Assemble: Object File Components

Text segment

```

00000000 <main>:
0:  ff010113  addi  sp  sp  -16
4:  00112623  sw    ra  12(sp)
8:  00000537  lui   a0  0x0
c:  00050513  addi  a0  a0  0
10: 000005b7  lui   a1  0x0
14: 00058593  addi  a1  a1  0
18: 000080e7  jalr  ra  0
1c: 00c12083  lw    ra  12(sp)
20: 01010113  addi  sp  sp  16
24: 00000513  addi  a0  a0  0
28: 00008067  jalr  ra
  
```

} la x2, call
 pseudo-
 instructions
 replaced

Address
placeholders
underlined



Read notes that describe these components of the object file!

Symbol Table

Label	Address (in module)	type
main:	0x00000000	global text
str1:	0x00000000	local data
str2:	0x0000000c	local data

Relocation Table

Address	type	Dependency
0x00000008	lui	%hi(str1)
0x0000000c	addi	%low(str1)
0x00000010	lui	%hi(str2)
...	...	...



Step 3. Link: `hello.o` → `a.out`

⋮

000101b0 <main>:

```
101b0: ff010113 addi sp sp -16
101b4: 00112623 sw ra 12(sp)
101b8: 00021537 lui a0 0x21
101bc: a1050513 addi a0 a0 -1520 # 20a10 <str1>
101c0: 000215b7 lui a1 0x21
101c4: a1c58593 addi a1 a1 -1508 # 20a1c <str2>
101c8: 288000ef jal ra 10450 # <printf>
101cc: 00c12083 lw ra 12(sp)
101d0: 01010113 addi sp sp,16
101d4: 00000513 addi a0 0,0
101d8: 00008067 jalr ra
```

⋮

- Squash all `.o` files
- Update in symbol table
- For each entry in the relocation table
 - Replace placeholders with the actual address
 - Update mach. code

[Aside] Static vs. Dynamic Linking

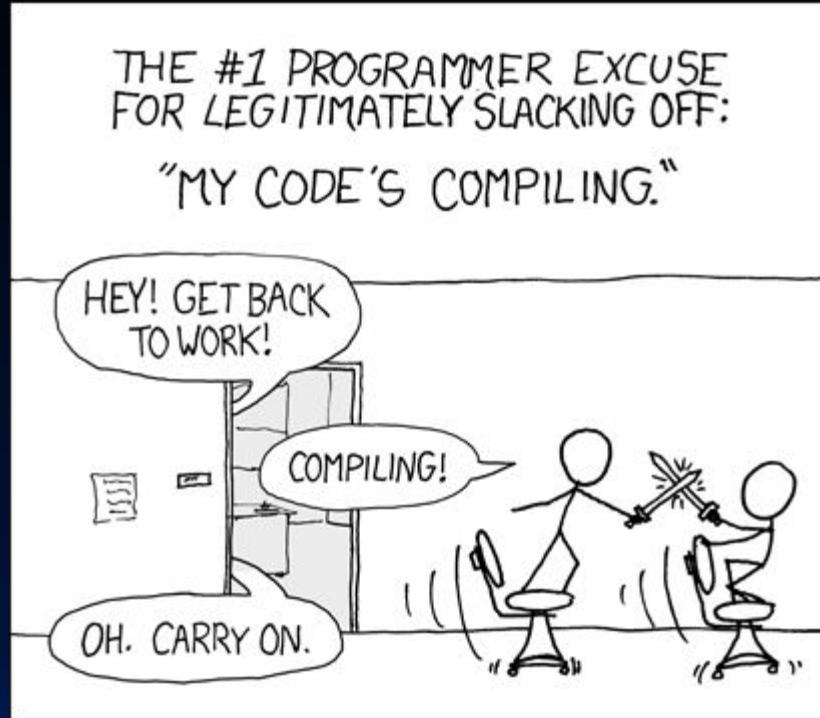


- CALL: Compile, Assemble, Link, Load
- [Example] Hello, World!
- [Aside] Static vs. Dynamically Linked Libraries

Aside (1/2): Static vs. Dynamic Linking

- So far, we've described the traditional way:
statically-linked libraries.
 - The library is now **part of the executable** – if the library updates, we won't get the fix unless we recompile the user program.
 - The executable includes the entire library, even if not all of it is used by the user program.
 - Pro: The Executable is self-contained!
- ✓ The Alternative: **dynamically-linked libraries (DLL)**, common on Windows & UNIX Platforms.

Statically Compiled Code



[xkcd](#), [explainxkcd](#)

Yan, SP26



Aside (2/2): Dynamically-Linked Libraries (DLLs)

Space vs. Time

- Storing a program requires less disk space; less time to send a program.
- Executing two programs requires less memory (if they share a library)
- ⚠ Runtime overhead: extra time to dynamically link!

Reasonable handling of upgrades

- Replacing `libXYZ.so` upgrades every program using library XYZ.
- ⚠ Having the program executable isn't enough anymore!

Prevailing approach to DLL uses machine code as the "lowest common denominator."

- "Linking at the machine code level,"
i.e., linker does not know what compiler/
what language compiled from.
- Not the only the way to do it...

Overall dynamic linking **adds complexity** to compiler, linker and OS. However, its **benefits outweigh its complexities.**

Yan, SFZb



And in Conclusion...

C program file: foo.c

Compiler

Assembly program file: foo.s

Assembler

Object file (mach. lang. module): foo.o

Linker

lib.o

Executable (mach. lang. program): a.out

Loader

Memory

- **Compiler** converts a single HLL file into a single assembly language file.
- **Assembler** removes pseudoinstructions, converts what it can to machine language, creates checklist for the linker (relocation table).
 - Does 2 passes to resolve addresses, handling internal forward references
- **Linker** combines several .o files and resolves absolute addresses.
 - Enables separate compilation, libraries that need not be compiled, and resolves remaining addresses
- **Loader** loads executable into memory and begins execution.

Yan, SP26



Agenda

**Extra Slides
[all in notes]**

- CALL: Compile, Assemble, Link, Load
- [Example] Hello, World!
- [Aside] Static vs. Dynamically Linked Libraries



Step 2: Assembler, Detailed

C program file: `foo.c`

Compiler

Assembly program file: `foo.s`

Assembler

Object file (mach. lang. module): `foo.o`

Linker

Executable (mach. lang. program) `foo.out`

Loader

Memory

- Input: Assembly Language Code (e.g., `foo.s` for RISC-V)
- Output: Machine Language Module, object file (e.g., `foo.o` for RISC-V)

- **Replace pseudoinstructions** with true assembly instructions.
- **Produce an object file** (a machine language **module**) that contains both this module's instructions and information for downstream steps (e.g., linking) and debugging.

[Text Segment] Produce Machine Code

- Arithmetic, Logical instructions
 - e.g., R-Format, some I-Format instructions
 - All necessary info to translate 32b representation is within the instruction!
- Resolve **position-independent code**
 - PC-Relative Branches and Jumps, e.g., e.g., beq/bne/etc. and jal
 - Once pseudoinstructions are replaced with real ones upstream (by Compiler), all **known** PC-relative addressing can be computed
 - Determine the offset to encode by counting the number of half-word instructions between current instruction and target instruction

```
add  x18 x18 x10
addi x19 x19 -1
```

```
j      Label
      ↕
jal    x0 Label
```

bne	rs1 rs2 label	if (rs1 != rs2) PC = PC + offset
jal	rd label	rd = PC + 4 PC = PC + offset
jalr	rd rs1 imm	rd = PC + 4 PC = rs1 + imm



[Text Segment] PC-Relative Addresses: 2-Pass

- The assembler performs **two passes** over the program to compute all offsets.
- “Forward Reference” problem:
 - Branches, PC-relative jumps can refer to labels that are **“forward”** in the program.
 - Precise **positive offsets** are unknown in the first pass!
- Instead, **take two passes** over the program.
 - Pass 1: Remember positions of labels (store in **symbol table**).
 - Pass 2: Use label positions to generate machine code.

```
Loop:  addi t2 zero 9
      slt  t1 zero t2
      beq  t1 zero Exit
      addi t2 t2 -1
      j   Loop
Exit: ...
```



forward jump



[Text Segment] Absolute Addresses? Not Yet

- References to **other files**?
 - e.g., `call(jalr)-ing strlen` from the C string library
- References to **static data**?
 - e.g., `la` gets broken up into `lui` and `addi`
 - These require knowing the full 32-bit address of the data
- **Absolute addresses** cannot be determined yet, so the Assembler records unresolved references in two tables:
 - Relocation Table and Symbol Table.



Assembler defers to downstream (Linker) any references that depend on knowing **other** object modules.

Directives: Information for Assembler

```
.text
.align 2
.globl main
main:
    addi sp,sp,-4
    sw   ra,0(sp)
    la   a0, str1
    la   a1, str2
    call printf
    lw   ra,0(sp)
    addi sp,sp,4
    li   a0,0
    ret
.section .rodata
.balign 4
str1:
.string "Hello, %s!\n"
str2:
.string "world"
```

Directives give directions to the assembler:

- Often generated by the compiler (previous stage)
- **Directives do not produce machine instructions!**
Rather, they inform for downstream how to build parts of the **output object file**.

<code>.text</code>	Put subsequent items in the user Text segment (machine code)
<code>...</code>	
<code>.data</code>	Put subsequent items in user Data segment (source file data in binary)
<code>...</code>	
<code>.globl sym</code>	Declares sym global and can be referenced from other files
<code>.string str</code>	Store the string str in memory and null-terminate it
<code>.word</code>	Store the n 32-bit quantities in successive memory words



Object File Format for This Object Module

1. **Object File Header**: size and position of other pieces of the object file
2. **Text Segment**: machine code
3. **Data Segment**: binary representation of static data in the source file
4. **Symbol Table**: List of file's labels, static data that can be referenced by other programs
5. **Relocation Table**: Lines of code to fix later (by Linker)
6. **Debugging Information**

"table of contents"

Resolves
PC-relative
addresses

For
downstream
use

Common Standard Format: ELF (UNIX)

http://www.skyfree.org/linux/references/ELF_Format.pdf

15-CALL (35)

[Assembler] Produce for Downstream

Symbol Table:

List of “items” in this object file

- Also used by debugger (gdb)!

Instruction labels

- Used to compute machine code for PC-relative addressing in branches, function calling, etc.
- Can use **.globl directive**: labels can be referenced by other files.

Data segment: anything in section marked by the **.data** directive

- Global variables may be accessed/used by other files.

Relocation Table:

List of “TODO items” whose address this file still needs

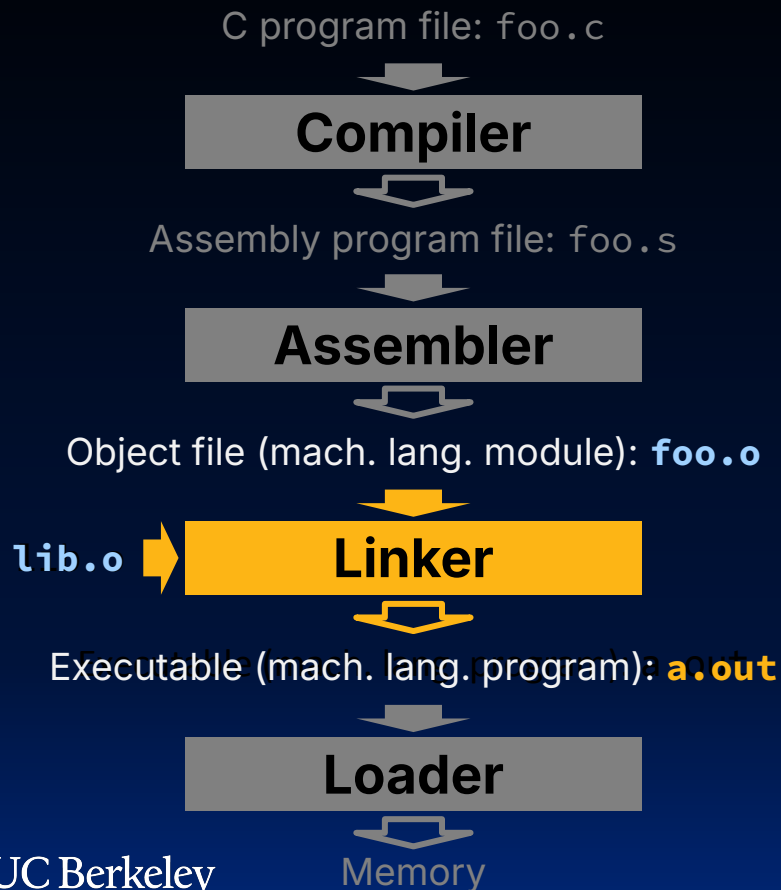
Any **external label** jumped to:

- e.g., in lib files
- `jal ext_label`

Any data located in **data segment**

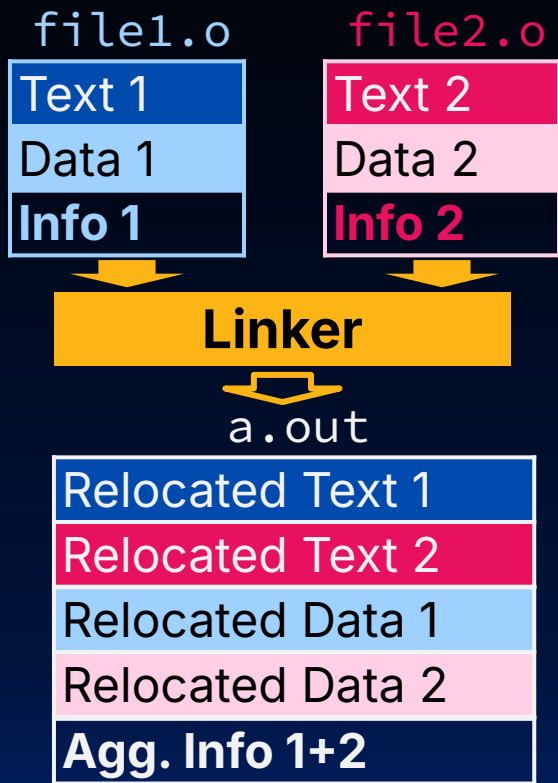
- e.g., static variables
- `la` load address instruction

Step 3: Linker, Detailed



- Input: Object files (e.g., `foo.o`, `lib.o` for RISC-V)
- Output: Executable machine code (e.g., `a.out` for RISC-V)
- The Linker enables **separate** compilation of files.
 - Changes to one file **does not require recompilation** of the entire program.
 - C standard libraries, e.g., `stdio` (e.g., Linux source >20M lines of code)

Linker Patches Together Multiple Object Modules



1. Put together **text segments** from each .o file.
2. Put together **data segments** from each .o file, then concatenate this onto the end of Step 1's segment.
3. Resolve references.
 - Go through **Relocation Table** and handle each entry, i.e., fill in/update all **absolute addresses** to external labels and static data.

The linker knows all input object file modules, so it can resolve all the assembler's "TODO items" that require relocating for the final executable.

Info: symbol table,
relocation table, etc.



Which Addresses Need Relocating?



- PC-Relative Addressing
 - `beq, bne, jal, auipc/addi`, etc.
- Already done by Assembler
 - Instruction offsets within single object module
- External Function Reference
 - Usually, `jal` or `auipc/jalr`
- Linker needs to relocate
 - Addresses to other object modules unknown at time of assembling
- Static Data Reference
 - `lw, sw` (of static data), `lui/addi` (when part of `la` pseudoinstr.)
- Linked needs to relocate
 - Data segment has now relocated as a result of multiple object modules



Resolving References (1/2)

- For RV32, linker assumes first text segment starts at address 0x10000.
 - (More later: virtual memory)
- Linker knows:
 - Length of each text and data segment
 - Ordering of text and data segments
- Linker calculates:
 - Absolute address of each label to be jumped to and of each piece of data referenced.

Resolving References (2/2)

- For RV32, linker assumes first text segment starts at address 0x10000.
 - (More later: virtual memory)
 - Linker knows:
 - Length of each text and data segment
 - Ordering of text and data segments
 - Linker calculates:
 - Absolute address of each label to be jumped to and of each piece of data referenced.
- Linker resolves references:
 - Search for reference (data or label) in all “user” symbol tables.
 - If not found, search library files (e.g., for printf).
 - Once absolute address determined, fill in machine code appropriately.
 - Linker Output:
 - Executable containing text and data along with header/debugging info)



Editing Instructions for Relocation

- **External jumps** (e.g., to C library):
 - J-Format
 - lui/addi; auipc/jalr (for li or la pseudoinstructions)
- Loads/stores that access **.data** variables
 - **Global pointer (gp)** is a pointer to the data (static) segment.
- (again, B-Type don't need editing, because these conditional jumps are within each module)

	31	26	24	20	19	15	14	12	11	7	6	0
J-/U-Type	xxxx xxxx xxxx xxxx xxxx xxxx										rd	opcode
I-Type	xxxx xxxx xxxx					rs1	funct3	rd		opcode		
S-Type	xxxx xxx			rs2		rs1	funct3	xxxxxx			opcode	



Loader...What does it do?

[Take CS162!]

Load program into a newly created address space:

- Read a .out file header for sizes of text, data segments.
- Create new address space for program large enough to hold **text and data segments**, along with a stack segment.
 - Copy instructions, data from executable file into new address space.
 - Copy arguments passed to the program onto the stack.

Initialize machine registers

- Most registers cleared; stack pointer (sp) assigned address of first free stack location

Jump to start-up routine:

- Copy program arguments from stack to registers, set PC

If main routine returns, terminate program with exit system call.



The **executable a.out** contains both machine code (**text segment**) and static data (**data segment**).